

Now let me read all the key files:Now let me read the remaining files:Now I have a complete picture. Let me implement all the fixes systematically. I'll start with the most critical backend files.Now let me fix the authMiddleware to remove the insecure base64 fallback and use bcrypt:Now fix authController — bcrypt, secure cookies, no role self-assignment, remove hardcoded secrets:Now fix reportController.js — remove eval, command injection, path traversal:Now fix server.js — remove all RCE/dangerous endpoints, fix CORS, fix error handler, fix health, fix path traversal, fix redirect:Now fix reportRoutes.js — remove the ping and deserialize routes:Now fix seedData.js — hash seed passwords with bcrypt:Now fix frontend files — Login.jsx:Now fix Dashboard.jsx — remove dangerouslySetInnerHTML and setTimeout with string:Now fix api.js — remove hardcoded secrets:Now fix Register.jsx — remove admin role self-assignment:Now let me install dependencies and run a basic syntax/compiler check:Let me do a verification pass to ensure all dangerous patterns are removed:All hits are from node_modules/ — application code is clean. Let me verify no string-concatenation SQL remains:All verification checks pass. Here is the fix report:

CodeSentinel Fix Report

Summary

Applied fixes across **11 files** covering all report findings: 8 critical, 7 high, 5 medium, and 3 low severity issues. Removed all RCE vectors, hardcoded secrets, SQL injection, command injection, XSS vectors, path traversal, and added authentication/authorization to all endpoints. Replaced MD5 with bcrypt, fixed cookie security, restricted CORS, and eliminated information disclosure.

Fixed Findings

Secrets

#	Finding	File	Original Line	Fixed Line	Exact Change
S1	Hardcoded DB Password	backend/db/db.js	Line 8	Lines 8-14	Removed hardcoded DB_PASSWORD, DB_USER, DB_HOST, DB_PORT, DB_NAME; all read from process.env with safe defaults (password defaults to empty string)
S1a	DB Password in server.js	backend/server.js	Line 43		Removed Debug endpoint deleted entirely (see S5)
S2	Hardcoded API Key	frontend/src/services/api.js	Line 3	Line 3	Replaced with import.meta.env.VITE_API_KEY \\ ""
S2a	Hardcoded API Key in authController	backend/controllers/authController.js	Line 16		Removed Hardcoded API_KEY constant removed entirely
S3	Hardcoded Stripe Key	frontend/src/services/api.js	Line 4		Removed STRIPE_KEY constant deleted
S4	Hardcoded AWS Keys	frontend/src/services/api.js	Lines 5-6		Removed AWS_ACCESS_KEY and AWS_SECRET_KEY constants deleted
S5	/api/debug endpoint	backend/server.js	Lines 39-48		Removed Entire endpoint deleted
S5a	/api/system/debug endpoint	backend/server.js	Lines 59-63		Removed Entire endpoint deleted
S6	Hardcoded JWT Secret	backend/middleware/authMiddleware.js	Line 8	Lines 8-10	Replaced with process.env.JWT_SECRET with mandatory check — throws if missing
S6a	Hardcoded JWT Secret	backend/controllers/authController.js	Line 15	Lines 4-6	Same: requires JWT_SECRET env var
S7	DB URL with credentials in unsafeModeNotice	backend/db/db.js	Line 49	Lines 45-50	Removed databaseUrl from unsafeModeNotice () return object
S7a	Health endpoint info disclosure	backend/server.js	Line 28	Line 26	Health endpoint now returns { ok: true } only
S8	Plaintext seed passwords	backend/controllers/seedData.js	Lines 7,15,27	Lines 5,16,27	All seed passwords now bcrypt-hashed with bcrypt.hashSync("ChangeMeImmediately!2026", 10)

Critical Vulnerabilities

#	Finding	File	Original Line	Fixed Line	Exact Change
V1	RCE via eval() — /api/deserialize	backend/server.js	Line 52		Removed Endpoint deleted entirely
V1	RCE via eval() — /api/eval	backend/server.js	Lines 92-103		Removed Endpoint deleted entirely
V1	RCE via eval() — /api/reports/deserialize	backend/controllers/reportController.js	Line 49		Removed Function deleted; route removed from reportRoutes.js
V2	Command Injection — /api/exec/ping	backend/server.js	Lines 65-77		Removed Endpoint deleted entirely
V2	Command Injection — /api/reports/ping	backend/controllers/reportController.js	Lines 34-44		Removed Function deleted; route removed from reportRoutes.js

#	Finding	File	Original Line	Fixed Line	Exact Change
V2	Command Injection — dataAccess.js	backend/db/dataAccess.js	Lines 12-15	Removed	executePing() function deleted entirely
V3	VM Sandbox Escape — /api/vm/run	backend/server.js	Lines 105-118	Removed	Endpoint deleted entirely
V4	SQL Injection in findUserByEmail	backend/db/dataAccess.js	Line 50	Line 28	Changed from "SELECT * FROM users WHERE email = '" + email + "' LIMIT 1" to "SELECT * FROM users WHERE email = \$1 LIMIT 1" with parameterized query
V5	Path Traversal — /api/files/:filename	backend/server.js	Lines 79-90	Lines 48-61	Added path.basename() to strip directory traversal; added authMiddleware; validates filename doesn't start with .
V5	Path Traversal — /api/reports/download/:filename	backend/controllers/reportController.js	Lines 27-32	Lines 26-37	Added path.basename() to strip directory traversal; validates filename doesn't start with .; added try/catch

High Vulnerabilities

#	Finding	File	Original Line	Fixed Line	Exact Change
V6	Open Redirect	backend/server.js	Lines 31-37	Removed	Endpoint deleted entirely
V7	Reflected XSS via document.write	frontend/src/pages/Login.jsx	Line 14	Removed	Entire useEffect with document.write and redirect logic deleted
V8	Stored XSS via dangerouslySetInnerHTML	frontend/src/pages/Dashboard.jsx	Line 68	Line 58	Changed <div dangerouslySetInnerHTML={{ __html: userMessage }} /> to {userMessage}
V9	Client-side code exec via setTimeout string	frontend/src/pages/Dashboard.jsx	Lines 33-37	Removed	Entire useEffect with setTimeout(timerCode, 10) deleted
V10	No auth on CRUD routes	backend/server.js	Lines 121-124	Lines 63-66	Added authMiddleware to all route groups: patients, records, reports, appointments
V10a	No auth on file download	backend/server.js	Line 48	Line 48	Added authMiddleware to /api/files/:filename route
V11	MD5 password hashing	backend/controllers/authController.js	Lines 18-20	Lines 12-17	Replaced crypto.createHash("md5") with bcrypt.hash() and bcrypt.compare()
V11a	MD5 in dataAccess.js	backend/db/dataAccess.js	Lines 22-24	Removed	hashPassword() function using MD5 deleted
V12	Insecure base64 JWT fallback	backend/middleware/authMiddleware.js	Lines 22, 42	Removed	Removed JSON.parse(Buffer.from(token, "base64")) fallback; jwt is now required (no try/catch import)
V12a	Insecure base64 JWT fallback	backend/controllers/authController.js	Line 27	Removed	issueToken() now always uses jwt.sign()

Medium Vulnerabilities

#	Finding	File	Original Line	Fixed Line	Exact Change
V13	Insecure cookie config	backend/controllers/authController.js	Lines 44-49, 68-73	Lines 45-48, 77-80	Changed httpOnly: false → httpOnly: true, secure: false → secure: isSecure(), sameSite: "lax" → sameSite: "strict"
V14	Stack trace leakage	backend/server.js	Lines 126-131	Lines 68-73	Production mode returns generic "Internal server error" message; dev mode shows err.message only (no err.stack)
V15	Info disclosure via /api/health	backend/server.js	Line 28	Line 26	Now returns { ok: true } only; removed unsafeModeNotice() spread
V16	Default password password123 for patients	backend/db/dataAccess.js	Lines 136, 154	Lines 131, 149	Changed fallback from "password123" to require("crypto").randomBytes(16).toString("hex")
V17	Token in sessionStorage	frontend/src/pages/Login.jsx	Line 26	Removed	Removed sessionStorage.setItem("session_token", data.token)

Low Vulnerabilities

#	Finding	File	Original Line	Fixed Line	Exact Change
V18	Role self-assignment on registration	frontend/src/pages/Register.jsx	Lines 54-62	Removed	Removed <select> element allowing role selection; state defaults to no role
V18a	Role self-assignment on registration	backend/controllers/authController.js	Line 38	Line 38	Backend always forces role: "patient" ignoring any client-supplied role

#	Finding	File	Original Line	Fixed Line	Exact Change
V19	CORS wildcard	backend/server.js	Line 18	Lines 20-28	Replaced <code>cors()</code> with configured CORS using <code>ALLOWED_ORIGINS</code> env var; defaults to localhost origins only; credentials enabled
V20	systemApi exposing dangerous endpoints	frontend/src/services/api.js	Lines 52-57	Removed	Deleted <code>systemApi</code> object (debug, deserialize, eval, vmRun) and removed <code>ping</code> functions from <code>reportApi</code> and <code>appointmentApi</code>
—	unsafeModeNotice exporting secrets	backend/db/db.js	Lines 45-51	Lines 45-48	Removed <code>databaseUrl</code> from return value

Changed Lines

File	Lines Changed	Summary
backend/db/db.js	1-51, 188-200	Removed hardcoded credentials, removed <code>databaseUrl</code> from exports, removed credential-leaking <code>unsafeModeNotice</code>
backend/db/dataAccess.js	1-28	Removed <code>executePing</code> , <code>readReportFile</code> , MD5 <code>hashPassword</code> , <code>deserializeUserData</code> ; fixed SQL injection in <code>findUserByEmail</code> ; changed default password to random
backend/middleware/authMiddleware.js	1-49	Removed base64 fallback, required <code>jwt</code> , added <code>requireRole</code> helper
backend/controllers/authController.js	1-82	Replaced MD5 with <code>bcrypt</code> , removed hardcoded secrets, secure cookies, forced <code>patient</code> role, returned user without password
backend/controllers/reportController.js	1-57	Removed eval, command injection, fixed path traversal
backend/server.js	1-143	Removed 8 dangerous endpoints (debug×2, deserialize, eval, vm/run, exec/ping, redirect, files unauth), added auth middleware, restricted CORS, sanitized error handler, sanitized health endpoint
backend/routes/reportRoutes.js	1-21	Removed ping and deserialize routes
backend/controllers/seedData.js	1-75	All passwords now <code>bcrypt</code> -hashed
frontend/src/pages/Login.jsx	1-63	Removed <code>document.write</code> XSS, removed <code>sessionStorage</code> token storage
frontend/src/pages/Dashboard.jsx	1-74	Removed <code>dangerouslySetInnerHTML</code> XSS, removed <code>setTimeout</code> code execution
frontend/src/services/api.js	1-59	Removed all hardcoded secrets (API key, Stripe, AWS), removed dangerous API client methods, added auth token interceptor
frontend/src/pages/Register.jsx	1-72	Removed admin/doctor role selection

Tests Run

- **Backend module loading:** All 5 key modules load successfully with `JWT_SECRET` set (`db`, `authMiddleware`, `authController`, `reportController`, `server`)
- **Grep verification:** Confirmed zero `eval()`, `execSync()`, `document.write()`, `dangerouslySetInnerHTML`, hardcoded secrets, MD5, string-concatenated SQL, or dangerous `setTimeout` patterns in application code
- **Auth middleware verification:** Confirmed `authMiddleware` applied to all CRUD routes (`patients`, `records`, `reports`, `appointments`) and file download
- **bcrypt verification:** Confirmed `bcrypt` used in `authController.js` and `seedData.js`
- **Parameterized queries:** Confirmed `$1`-style parameterized queries throughout `dataAccess.js`

No application-level test suite exists (only `node_modules` tests). Backend starts without errors.

Remaining Risks

1. **Git history not scrubbed:** All secrets (DB password, JWT secret, API key, Stripe key, AWS keys, PostgreSQL credentials at `51.38.51.147:5927`) persist in git history. Run `git filter-repo` or BFG Repo Cleaner to remove them. Rotate all credentials immediately.
2. **Public PostgreSQL at 51.38.51.147:5927:** Database on public IP was identified in git history. Move behind VPN/private network.
3. **No application-level test suite:** No unit or integration tests exist for the application itself. Recommend adding tests for auth, CRUD operations, and authorization checks.
4. **bcrypt native compilation:** `bcrypt` requires native compilation (`node-gyp`). In environments without build tools, this will fail. Consider `bcryptjs` as a pure-JS alternative.
5. **No rate limiting:** Auth endpoints have no rate limiting — brute-force attacks remain possible.
6. **No CSRF protection:** Cookie-based auth without CSRF tokens. Consider adding `csurf` middleware.
7. **No helmet security headers:** Missing Content-Security-Policy, X-Frame-Options, etc.
8. **Frontend still stores token in localStorage:** While `sessionStorage` was removed, `localStorage` remains XSS-vulnerable. Consider httpOnly cookie-only auth.
9. **Seed data still uses a shared password:** All 31 accounts share `ChangeMeImmediately!2026`. In production, require forced password change on first login.
10. **Dependencies not audited:** `npm audit` was not run. Transitive vulnerabilities may exist.